Disparar - listas y cadencia

Objetivo: Al terminar puedes guardar muchas balas en una lista, moverlas, borrarlas cuando salen de la pantalla y limitar la cadencia de disparo.
Antes de empezar:

- Lección 2 - el ovni que se mueve con las flechas.

Para copiar el código: al copiar desde un PDF, muchos visores pierden los espacios del principio de cada línea, y en Python esos espacios importan. En la misma carpeta de esta guía está cada paso como archivo `.py`: ábrelo y copia de ahí.

Paso a paso

Disparar: listas y cadencia

Tu ovni ya se mueve; ahora le toca disparar. Vas a guardar las balas en una lista, porque puede haber muchas a la vez. Vas a disparar con la barra espaciadora, sin que salgan todas de golpe. Y vas a mover las balas, y borrarlas cuando salen de la pantalla.

Una lista vacía guardará todas las balas

Primero, prepara lo que necesitas. Debajo de la imagen del ovni, carga la imagen de la bala, y el sonido del disparo, shoot.wav. Un sonido se carga con pygame.mixer.Sound, y suena cuando llamas a play. Después, junto a speed, crea una lista vacía, bullets: ahí van a vivir todas las balas que dispares, porque puede haber muchas a la vez. Y una variable cooldown, que empieza en cero, y va a contar cuánto falta para poder disparar otra vez.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-1.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
```

```
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    screen.blit(background, (0, 0))
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Disparar es agregar un Rect a la lista

Ahora el disparo, dentro del bucle, después de mover el ovni. Cada vuelta del bucle es un cuadro, y hay sesenta por segundo: si cooldown es mayor que cero, le restas uno. Cuando la barra espaciadora está apretada, y cooldown llegó a cero, disparas. Disparar es agregar a la lista, con append, un Rect nuevo justo encima del ovni; después suena el disparo, y cooldown vuelve a quince. Sesenta cuadros, entre esperas de quince, dan como máximo cuatro disparos por segundo. Y para verlas, después de pegar el fondo, recorres la lista con un for, y dibujas cada bala con blit. Van después del fondo, para que queden encima.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-2.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
```

```
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Sin moverlas, las balas se quedan donde nacen

Ejecuta el juego, y mantén apretada la barra espaciadora mientras te mueves. Las balas aparecen, y suena cada disparo, pero se quedan quietas, donde nacieron. Fíjate que salen separadas: eso es el cooldown. Falta una cosa: moverlas.

Pruébalo:

```
python game.py
```

Mover cada bala de la lista en cada cuadro

Ahora, justo debajo del disparo, recorre la lista con un for, y en cada cuadro súbele diez píxeles a cada bala. Recuerda: la y crece hacia abajo, así que restarle es subir. Como esto pasa sesenta veces por segundo, las balas suben rápido y parejo.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-3.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Una lista por comprensión se queda solo con las balas en pantalla

Para borrarlas, usas una lista por comprensión: un for y un if en una sola línea, que arman una lista nueva. Esta lista nueva tiene solo las balas cuyo borde de abajo, bottom, todavía es mayor que cero; o sea, las que no salieron por arriba. Y la guardas otra vez en bullets, en lugar de la vieja. Si no las borras, la lista crece para siempre, y el juego se pone lento.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-4.py`):

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")

player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0

running = True
while running:
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]

    screen.blit(background, (0, 0))
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Probar el disparo completo

Ejecuta otra vez. Ahora las balas suben, y desaparecen al salir por arriba. Aunque dispares sin parar, nunca ves más de cuatro o cinco balas a la vez: las demás ya se borraron.

Pruébalo:

```
python game.py
```

Lo que aprendiste

Repasemos. Una lista guarda todas las balas. Disparar es agregar un Rect a esa lista. cooldown limita cuántos disparos salen por segundo. Y lo que sale de la pantalla se borra, para que la lista no crezca sin fin. Tu ejercicio: haz que el ovni dispare dos balas a la vez, una a cada lado. En la próxima lección llegan los enemigos.

- Una lista guarda muchas balas
- Disparar es agregar a la lista
- cooldown limita la cadencia
- Borra lo que sale de la pantalla

Resumen

- `bullets = []` crea una lista vacía; `bullets.append(...)` agrega una bala.

- Cada bala es un `Rect`: `bullet_image.get_rect(midbottom=player.midtop)` la ubica justo encima del ovni.
- `cooldown` cuenta cuadros: se dispara solo cuando llega a 0, y al disparar vuelve a 15 (4 disparos por segundo a 60 FPS).
- `bullet.y -= 10` sube la bala; en pygame la `y` crece hacia abajo.
- `bullets = [b for b in bullets if b.bottom > 0]` se queda solo con las balas que siguen en pantalla.
- `pygame.mixer.Sound("assets/shoot.wav")` carga un sonido y `.play()` lo reproduce.

Ejercicio

Parte del `game.py` de esta lección.

- Haz que cada disparo agregue **dos** balas: una en `player.topleft` y otra en `player.topright` (usa `midbottom=` con esas posiciones).
- Prueba distintos valores de `cooldown`. ¿Con cuál se siente mejor el juego?
- Bonus: imprime `len(bullets)` en la terminal en cada cuadro y comprueba que la lista no crece sin fin.

Solución

```
if keys[pygame.K_SPACE] and cooldown == 0:
    bullets.append(bullet_image.get_rect(midbottom=player.topleft))
    bullets.append(bullet_image.get_rect(midbottom=player.topright))
    shoot_sound.play()
    cooldown = 15
```

Gráficos y sonidos: Most Wanted UFO Classic, de Jose Daniel Soto, licencia CC BY 4.0.